

Seguridad de la Información

Práctica 3: Cifrado asimétrico

TITULACIÓN: Grado en Ingeniería Informática

AUTOR: Sergio Acosta Quintana

Fecha: 1 de marzo de 2026

Índice general

1. Introducción y Configuración Previa	2
2. Generación de claves asimétricas (pública-privada), firmado de resúmenes (RSA y DSA) y derivación de secretos con claves DH y ECDH (X25519)	3
2.1. Claves RSA: Generación, exportación y firma	3
2.2. Claves DSA	6
2.3. Derivación de secretos DH Estándar	7
2.4. Derivación con curva elíptica X25519	9
3. Intercambio de información segura (cifrado asimétrico de clave simétrica y firma)	11
3.1. Generación de claves de Ana y Berto	11
3.2. Operaciones de Ana (Emisor)	12
3.3. Operaciones de Berto (Receptor)	14

Capítulo 1

Introducción y Configuración Previa

En este documento se detalla la resolución de los ejercicios correspondientes a la tercera práctica de Seguridad de la Información. A lo largo de las siguientes secciones se documenta el manejo de la herramienta OpenSSL para la realización de operaciones criptográficas relativas al cifrado asimétrico, la generación y gestión de claves, el cifrado de resúmenes y el cifrado de claves simétricas.

Como paso previo para el correcto desarrollo, se ha preparado un entorno de trabajo dentro de una carpeta llamada `practica3`. Se ha personalizado el prompt de la consola ejecutando `export PS1="Sergio Acosta > "` para identificar adecuadamente las capturas de pantalla. Seguidamente, y cumpliendo con las indicaciones del enunciado, hemos reutilizado uno de los ficheros de texto de la práctica anterior copiándolo y renombrándolo a `texto.txt`.

Capítulo 2

Generación de claves asimétricas (pública-privada), firmado de resúmenes (RSA y DSA) y derivación de secretos con claves DH y ECDH (X25519)

2.1. Claves RSA: Generación, exportación y firma

El primer algoritmo empleado es RSA. Como podemos ver en la figura 2.1, se ejecutan los comandos para generar un par de claves asimétricas de 2048 bits en formato PEM. Se utiliza la opción `-aes256` para proteger la clave privada mediante una contraseña. Posteriormente, se extrae la clave pública a un fichero independiente.

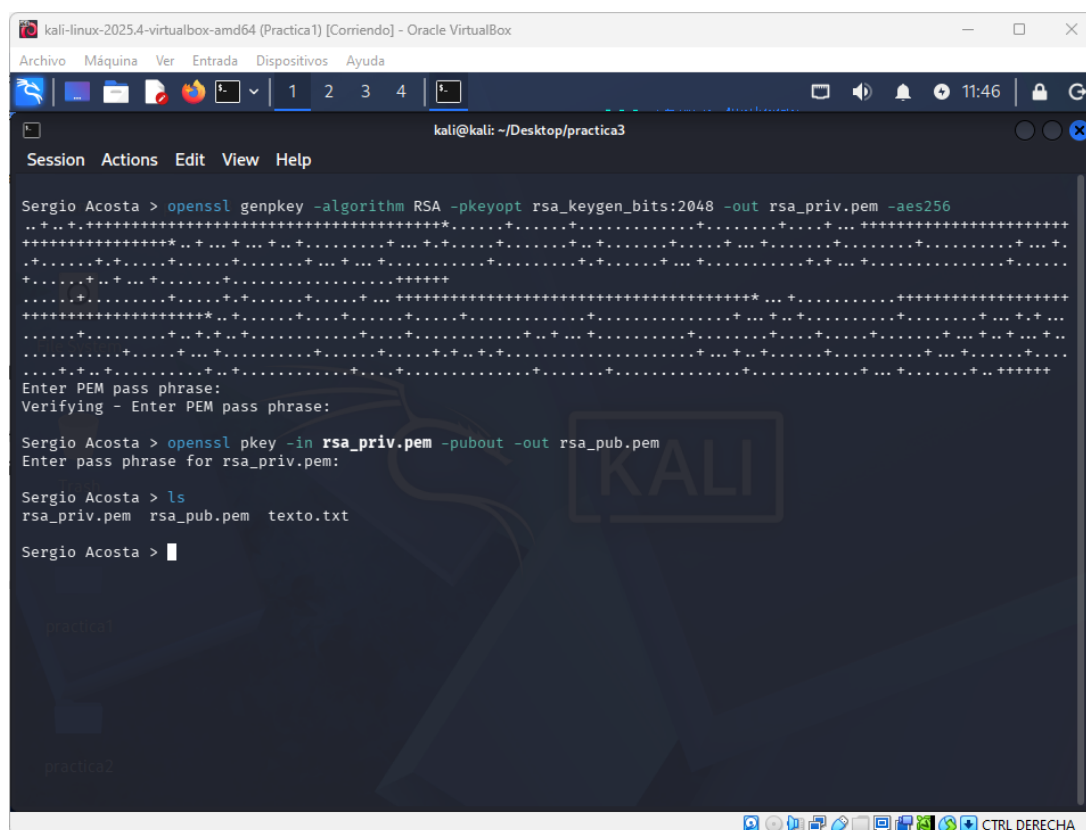
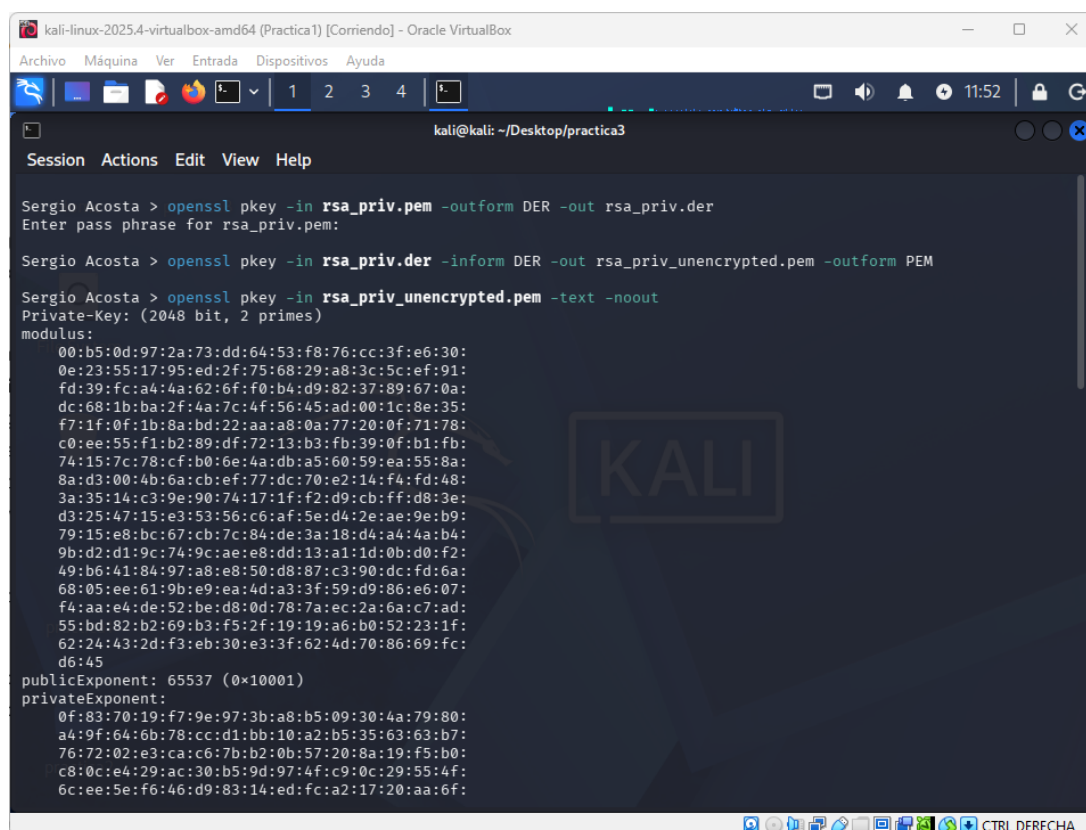


Figura 2.1: Generación de la clave privada protegida por contraseña y extracción de la clave pública.

A continuación, se exporta la clave privada a formato DER, que consiste en código binario puro, y de vuelta a formato PEM. Como podemos comprobar en la figura 2.2, al realizar esta conversión, la herramienta elimina por defecto el cifrado. De esta forma, la clave queda totalmente desprotegida y se pueden visualizar sus componentes matemáticos en la consola sin necesidad de proporcionar ninguna contraseña.



```
kali@kali: ~/Desktop/practica3
Session Actions Edit View Help

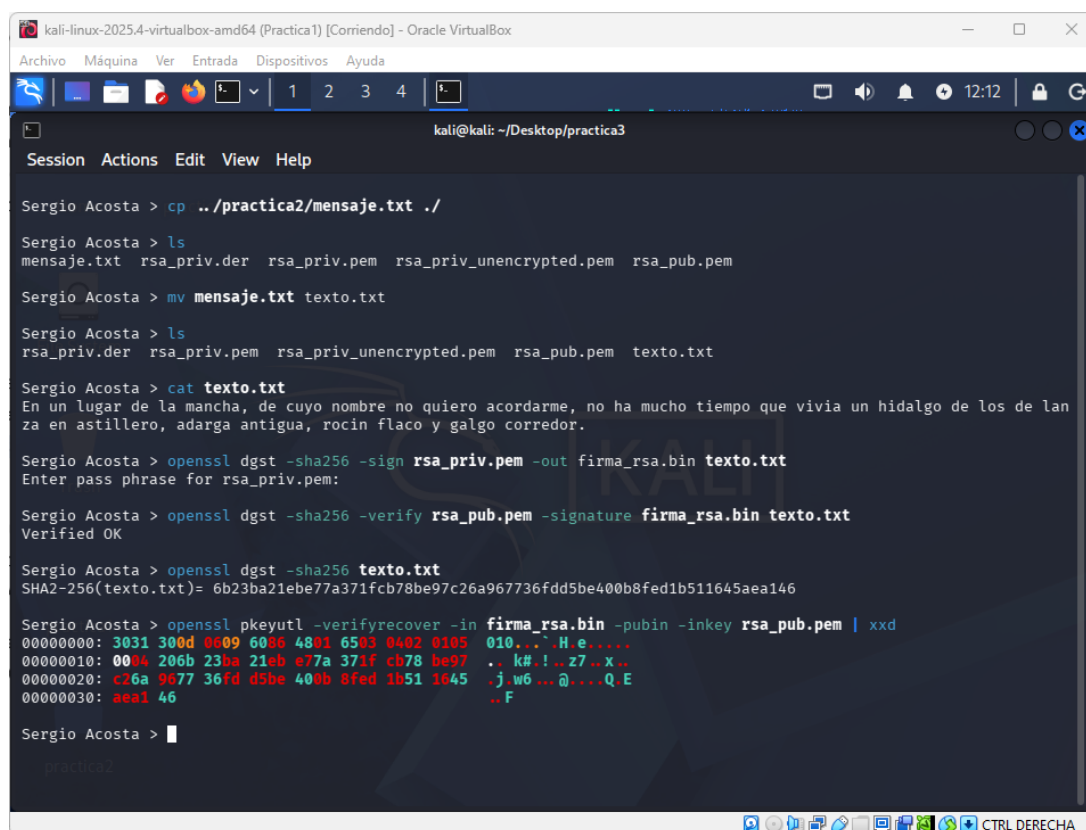
Sergio Acosta > openssl pkey -in rsa_priv.pem -outform DER -out rsa_priv.der
Enter pass phrase for rsa_priv.pem:

Sergio Acosta > openssl pkey -in rsa_priv.der -inform DER -out rsa_priv_unencrypted.pem -outform PEM

Sergio Acosta > openssl pkey -in rsa_priv_unencrypted.pem -text -noout
Private-Key: (2048 bit, 2 primes)
modulus:
 00:b5:0d:97:2a:73:dd:64:53:f8:76:cc:3f:e6:30:
 0e:23:55:17:95:ed:2f:75:68:29:a8:3c:5c:ef:91:
 fd:39:fc:a4:4a:62:6f:f0:b4:d9:82:37:89:67:0a:
 dc:68:1b:ba:2f:4a:7c:4f:56:45:ad:00:1c:8e:35:
 f7:1f:0f:1b:8a:bd:22:aa:a8:0a:77:20:0f:71:78:
 c0:ee:55:f1:b2:89:df:72:13:b3:fb:39:0f:b1:fb:
 74:15:7c:78:cf:b0:6e:4a:db:a5:60:59:ea:55:8a:
 8a:d3:00:4b:6a:cb:ef:77:dc:70:e2:14:f4:fd:48:
 3a:35:14:c3:9e:90:74:17:1f:f2:d9:cb:ff:d8:3e:
 d3:25:47:15:e3:53:56:c6:af:5e:d4:2e:ae:9e:b9:
 79:15:e8:bc:67:cb:7c:84:de:3a:18:d4:a4:4a:b4:
 9b:d2:d1:9c:74:9c:ae:e8:dd:13:a1:1d:0b:d0:f2:
 49:b6:41:84:97:a8:e8:50:d8:87:c3:90:dc:fd:6a:
 68:05:ee:61:9b:e9:ea:4d:a3:3f:59:d9:86:e6:07:
 f4:aa:e4:de:52:be:d8:0d:78:7a:ec:2a:6a:c7:ad:
 55:bd:82:b2:69:b3:f5:2f:19:19:a6:b0:52:23:1f:
 62:24:43:2d:f3:eb:30:e3:3f:62:4d:70:86:69:fc:
 d6:45
publicExponent: 65537 (0x10001)
privateExponent:
 0f:83:70:19:f7:9e:97:3b:a8:b5:09:30:4a:79:80:
 a4:9f:64:6b:78:cc:d1:bb:10:a2:b5:35:63:63:b7:
 76:72:02:e3:ca:c6:7b:b2:0b:57:20:8a:19:f5:b0:
 c8:0c:e4:29:ac:30:b5:9d:97:4f:c9:0c:29:55:4f:
 6c:ee:5e:f6:46:d9:83:14:ed:fc:a2:17:20:aa:6f:
```

Figura 2.2: Conversión a formato DER y demostración de la desprotección de la clave privada.

Para finalizar con las operaciones de RSA, se procede a firmar un resumen del fichero original. Como vemos en la figura 2.3, se obtiene el resumen SHA-256 y se cifra con la clave privada. Seguidamente se verifica la firma con la clave pública, resultando en "Verified OK". Adicionalmente, se ejecuta una comprobación de manera manual con los parámetros `-verifyrecover` y `xxd` para demostrar visualmente cómo el hash extraído de la firma coincide a la perfección con el calculado de forma local.



```
kali@kali: ~/Desktop/practica3
Session Actions Edit View Help

Sergio Acosta > cp ../practica2/mensaje.txt ./
Sergio Acosta > ls
mensaje.txt  rsa_priv.der  rsa_priv.pem  rsa_priv_unencrypted.pem  rsa_pub.pem
Sergio Acosta > mv mensaje.txt texto.txt
Sergio Acosta > ls
rsa_priv.der  rsa_priv.pem  rsa_priv_unencrypted.pem  rsa_pub.pem  texto.txt
Sergio Acosta > cat texto.txt
En un lugar de la Mancha, de cuyo nombre no quiero acordarme, no ha mucho tiempo que vivia un hidalgo de los de lanza en astillero, adarga antigua, rocin flaco y galgo corredor.
Sergio Acosta > openssl dgst -sha256 -sign rsa_priv.pem -out firma_rsa.bin texto.txt
Enter pass phrase for rsa_priv.pem:
Sergio Acosta > openssl dgst -sha256 -verify rsa_pub.pem -signature firma_rsa.bin texto.txt
Verified OK
Sergio Acosta > openssl dgst -sha256 texto.txt
SHA2-256(texto.txt)= 6b23ba21ebe77a371fcb78be97c26a967736fdd5be400b8fed1b511645aea146
Sergio Acosta > openssl pkeyutl -verifyrecover -in firma_rsa.bin -pubin -inkey rsa_pub.pem | xxd
00000000: 3031 300d 0609 6086 4801 6503 0402 0105  010...`H.e.....
00000010: 0004 206b 23ba 21eb e77a 371f cb78 be97  ..k#!..z7..x..
00000020: c26a 9877 36fd d5be 400b 8fed 1b51 1645  .j.w6...@....Q.E
00000030: aea1 46                                     ..F
Sergio Acosta > 
```

Figura 2.3: Firma del resumen del texto y verificación correcta con la clave pública.

2.2. Claves DSA

A diferencia del algoritmo anterior, DSA se utiliza exclusivamente para firmas digitales. Como podemos comprobar en la figura 2.4, se repiten los pasos anteriores adaptados a claves DSA. Primero se generan los parámetros, luego se crean las claves protegidas, se demuestra la desprotección al pasarlo por DER y, por último, se realiza el firmado y la verificación del documento con éxito.

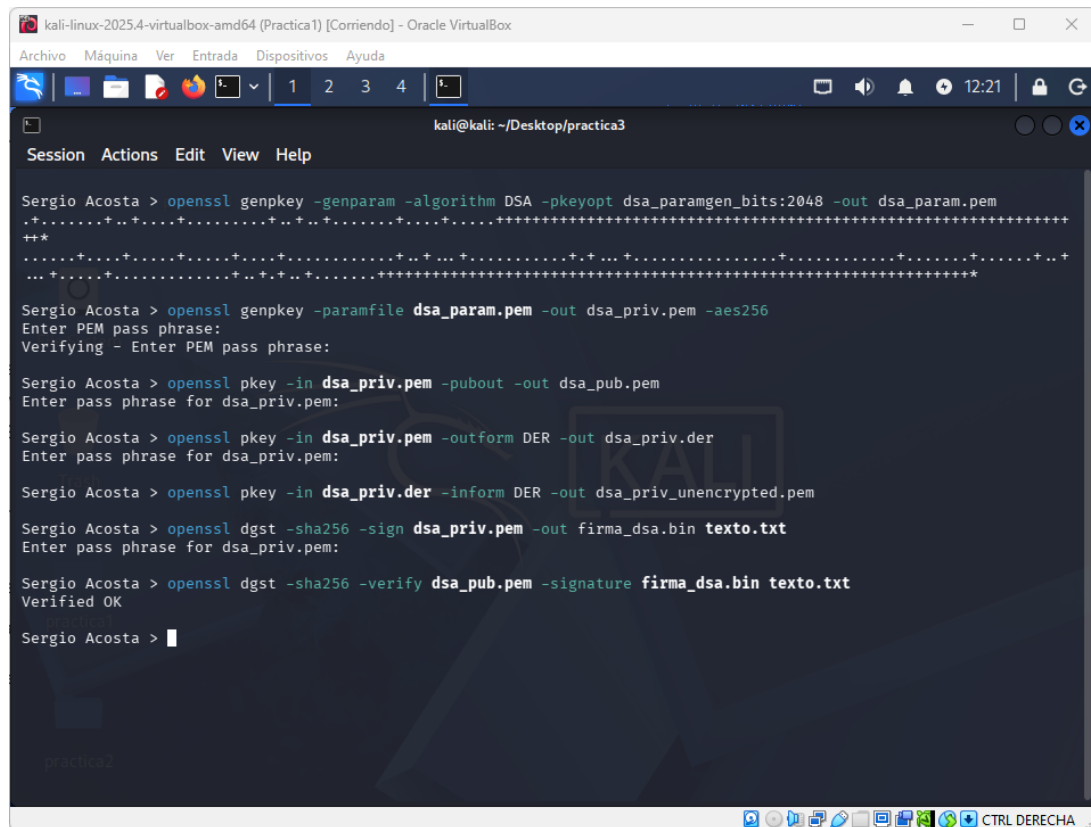


Figura 2.4: Generación de parámetros, claves, desprotección DER, firma y verificación con DSA.

2.3. Derivación de secretos DH Estándar

Para que dos agentes puedan acordar un secreto común a través de un canal inseguro, se utiliza el protocolo Diffie-Hellman. En primer lugar, como vemos en la figura 2.5, se generan los parámetros matemáticos base de 2048 bits, un proceso que requiere de cierta carga computacional.

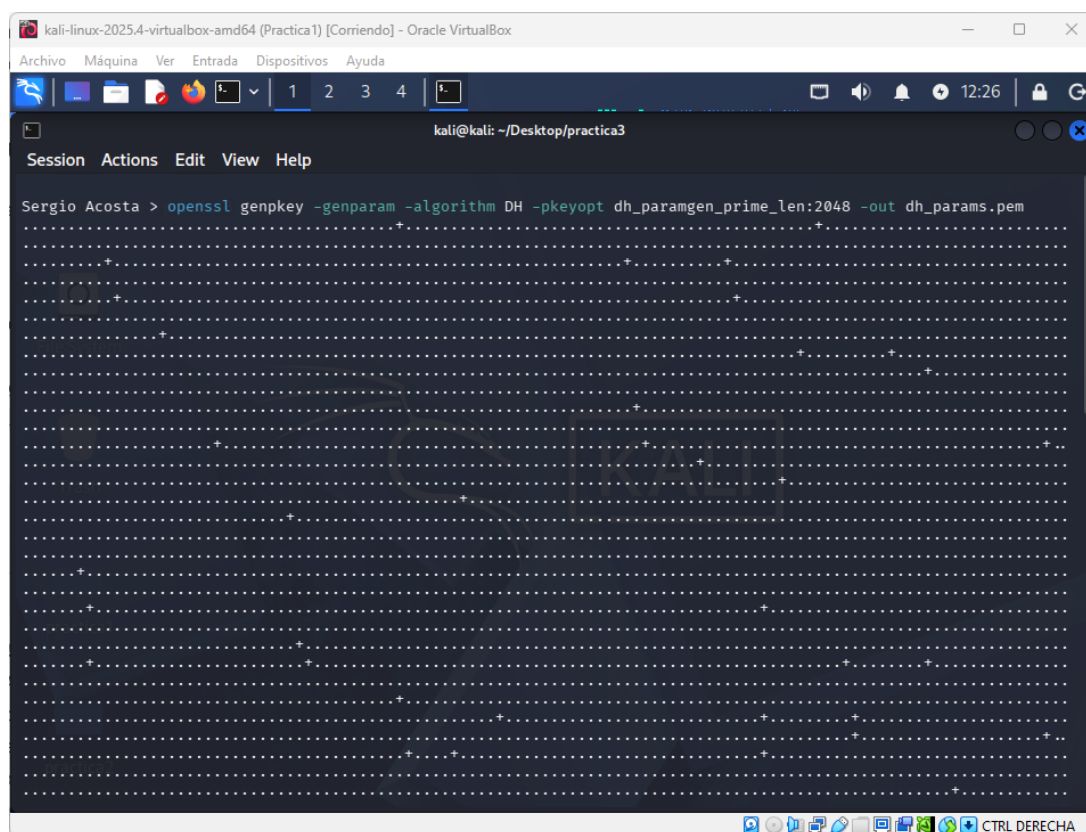
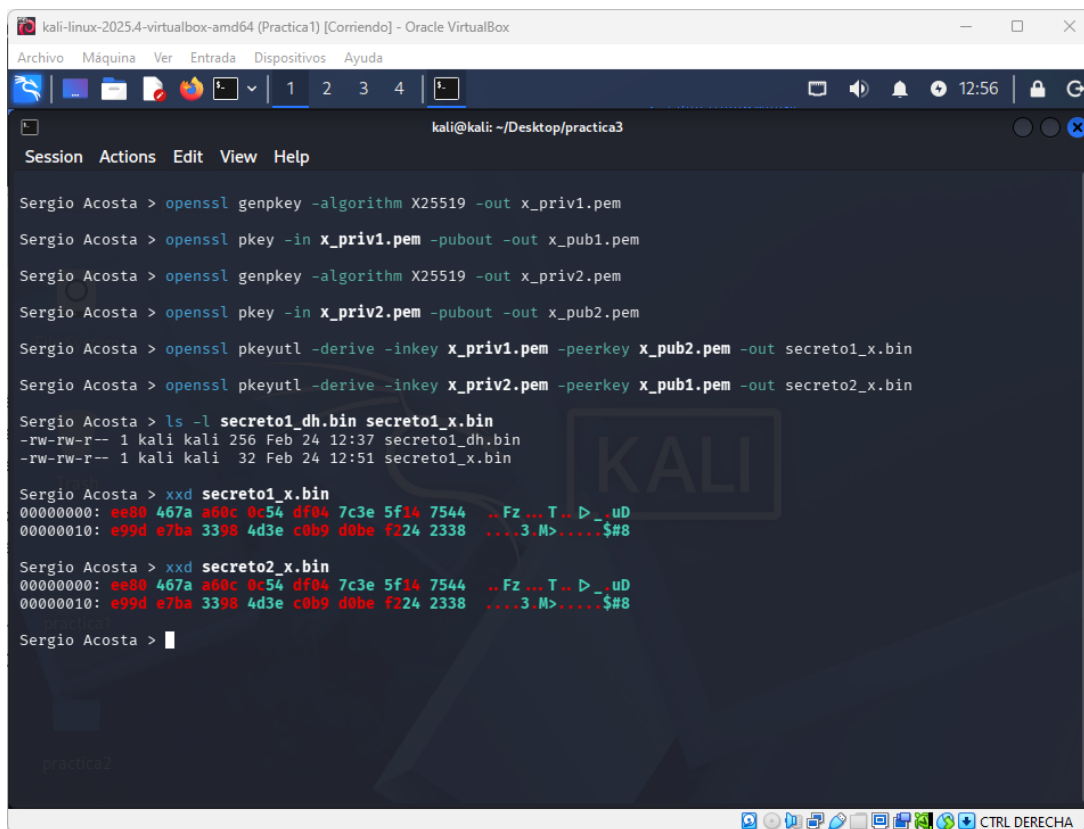


Figura 2.5: Generación de los parámetros matemáticos comunes para Diffie-Hellman.

A partir de estos parámetros, como podemos ver en la figura 2.6, se generan los pares de claves de ambos usuarios. Mezclando la clave privada de uno con la pública del otro se derivan los secretos cruzados. Mediante el comando `cmp` se comprueba que la salida es nula, lo cual demuestra que ambos ficheros binarios generados son matemáticamente idénticos.



```
kali@kali: ~/Desktop/practica3
Session Actions Edit View Help

Sergio Acosta > openssl genpkey -algorithm X25519 -out x_priv1.pem
Sergio Acosta > openssl pkey -in x_priv1.pem -pubout -out x_pub1.pem
Sergio Acosta > openssl genpkey -algorithm X25519 -out x_priv2.pem
Sergio Acosta > openssl pkey -in x_priv2.pem -pubout -out x_pub2.pem
Sergio Acosta > openssl pkeyutl -derive -inkey x_priv1.pem -peerkey x_pub2.pem -out secreto1_x.bin
Sergio Acosta > openssl pkeyutl -derive -inkey x_priv2.pem -peerkey x_pub1.pem -out secreto2_x.bin

Sergio Acosta > ls -l secreto1_dh.bin secreto1_x.bin
-rw-rw-r-- 1 kali kali 256 Feb 24 12:37 secreto1_dh.bin
-rw-rw-r-- 1 kali kali 32 Feb 24 12:51 secreto1_x.bin

Sergio Acosta > xxd secreto1_x.bin
00000000: ee80 467a a60c 0c54 df04 7c3e 5f14 7544  ..Fz...T..>_uD
00000010: e99d e7ba 3398 4d3e c0b9 d0be f224 2338  ....3.M>.....$#8

Sergio Acosta > xxd secreto2_x.bin
00000000: ee80 467a a60c 0c54 df04 7c3e 5f14 7544  ..Fz...T..>_uD
00000010: e99d e7ba 3398 4d3e c0b9 d0be f224 2338  ....3.M>.....$#8

Sergio Acosta >
```

Figura 2.7: Generación de secretos X25519 y comparativa de peso mediante `ls -l`.

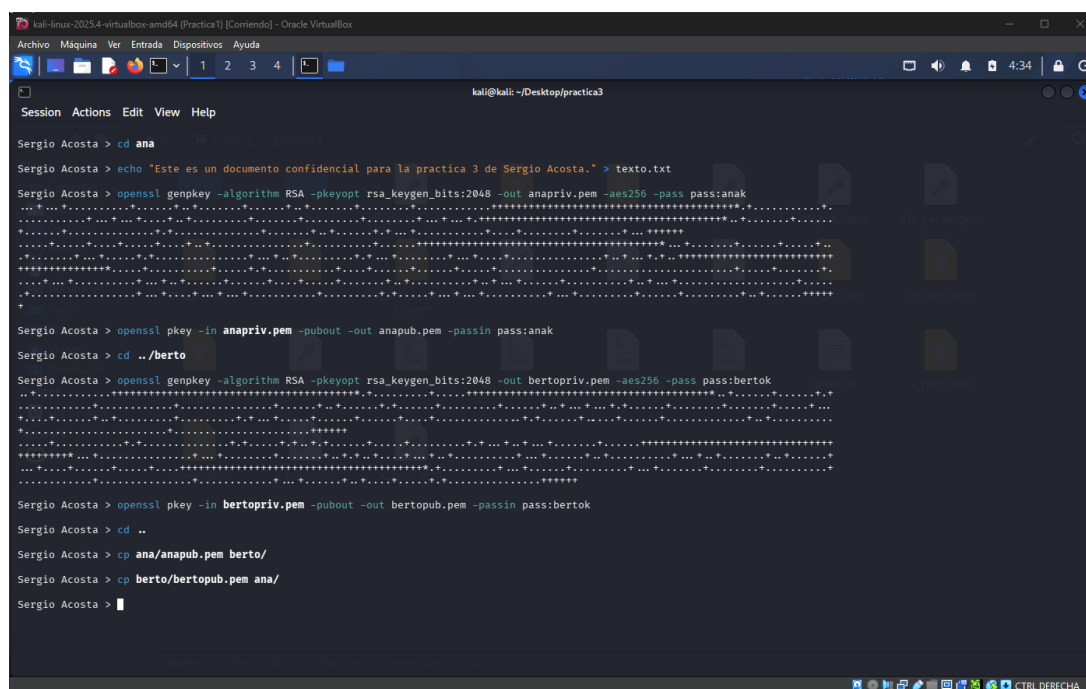
Capítulo 3

Intercambio de información segura (cifrado asimétrico de clave simétrica y firma)

En este apartado se reproduce, en una secuencia de operaciones real, el intercambio de información segura entre dos agentes utilizando cifrado simétrico, cifrado asimétrico de las claves simétricas y firma digital. Para emular el entorno de trabajo por separado, se han creado dos carpetas para los agentes Ana y Berto.

3.1. Generación de claves de Ana y Berto

Como tarea preparatoria, se generan las parejas de claves RSA de 2048 bits correspondientes a cada agente. Como podemos ver en la figura 3.1, se protegen las claves privadas con las contraseñas "anak" y "bertok". Para simular el envío de información, se copian y cruzan los archivos correspondientes a las claves públicas entre las carpetas de ambos.



```
kali@kali: ~/Desktop/practica3
Sergio Acosta > cd ana
Sergio Acosta > echo "Este es un documento confidencial para la practica 3 de Sergio Acosta." > texto.txt
Sergio Acosta > openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out anapriv.pem -aes256 -pass pass:anak
.....
Sergio Acosta > openssl pkey -in anapriv.pem -pubout -out anapub.pem -passin pass:anak
Sergio Acosta > cd ../berto
Sergio Acosta > openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out bertopriv.pem -aes256 -pass pass:bertok
.....
Sergio Acosta > openssl pkey -in bertopriv.pem -pubout -out bertopub.pem -passin pass:bertok
Sergio Acosta > cd ..
Sergio Acosta > cp ana/anapub.pem berto/
Sergio Acosta > cp berto/bertopub.pem ana/
Sergio Acosta >
```

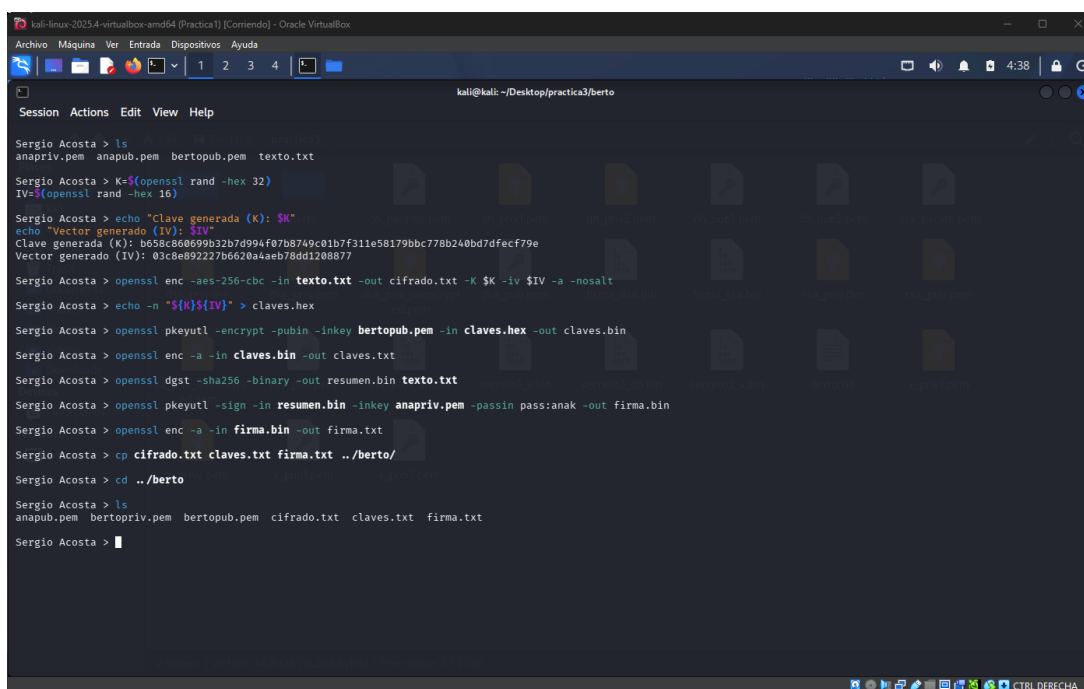
Figura 3.1: Generación de los pares de claves RSA y simulación de intercambio de claves públicas.

3.2. Operaciones de Ana (Emisor)

A continuación, se llevan a cabo las tareas del lado de Ana. Como podemos observar en la figura 3.2, primero se genera una clave y un vector de forma aleatoria y se almacenan en variables temporales de la sesión.

Utilizando esos datos, se cifra el fichero de texto con AES-256 en modo CBC. Se especifica que no se añada sal con el flag `-nosalt` y se exporta el resultado en formato BASE64 bajo el nombre `cifrado.txt`. Acto seguido, se concatenan la clave y el vector en un archivo y se somete a un cifrado asimétrico usando la clave pública de Berto, transformando el documento binario resultante a BASE64 en el archivo `claves.txt`. Para terminar, se firma el resumen SHA-256 binario del fichero de texto con la clave privada de Ana, guardando la salida en BASE64 en `firma.txt`.

Una vez preparados los archivos, se copian a la ruta de la carpeta de Berto.



```
kali@kali: ~/Desktop/practica3/berto
Sergio Acosta > ls
anapriv.pem  bertopub.pem  texto.txt

Sergio Acosta > K=$(openssl rand -hex 32)
IV=$(openssl rand -hex 16)

Sergio Acosta > echo "Clave generada (K): $K"
Clave generada (K): b658c860699b32b7d994f07b8749c01b7f311e58179bbc778b240bd7dfecf79e
Sergio Acosta > echo "Vector generado (IV): $IV"
Vector generado (IV): 03c8e89227b6620a4aeb78dd1208877

Sergio Acosta > openssl enc -aes-256-cbc -in texto.txt -out cifrado.txt -K $K -iv $IV -a -nosalt

Sergio Acosta > echo -n "${K}${IV}" > claves.hex

Sergio Acosta > openssl pkeyutl -encrypt -pubin -inkey bertopub.pem -in claves.hex -out claves.bin

Sergio Acosta > openssl enc -a -in claves.bin -out claves.txt

Sergio Acosta > openssl dgst -sha256 -binary -out resumen.bin texto.txt

Sergio Acosta > openssl pkeyutl -sign -in resumen.bin -inkey anapriv.pem -passin pass:anak -out firma.bin

Sergio Acosta > openssl enc -a -in firma.bin -out firma.txt

Sergio Acosta > cp cifrado.txt claves.txt firma.txt ../berto/

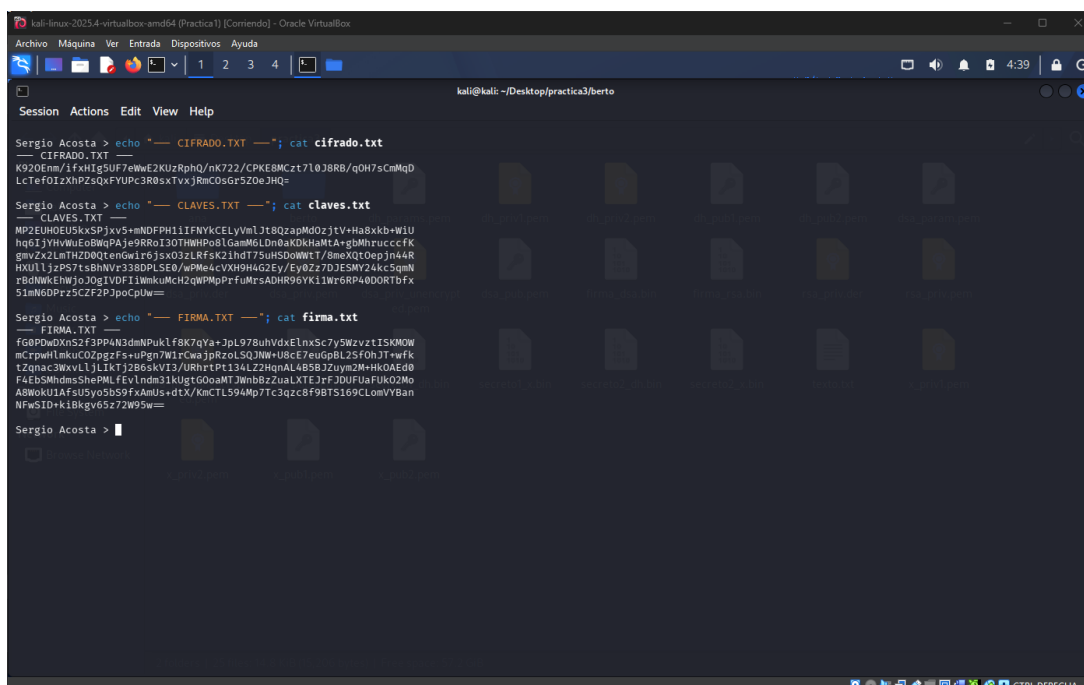
Sergio Acosta > cd ../berto

Sergio Acosta > ls
anapub.pem  bertopriv.pem  bertopub.pem  cifrado.txt  claves.txt  firma.txt

Sergio Acosta >
```

Figura 3.2: Cifrado simétrico del fichero, cifrado asimétrico de las claves y creación de la firma.

Cumpliendo con las indicaciones documentales de la práctica, en la figura 3.3 se muestran los listados por pantalla en formato BASE64 de los tres archivos enviados.



```
kali@kali: ~/Desktop/practica3/berto
Sergio Acosta > echo "==== CIFRADO.TXT ===="; cat cifrado.txt
==== CIFRADO.TXT ====
K920Em/1fXhIgsUf7eWwE2KUzRphq/nk722/CPKE8MCzt7l0J8RB/qOH7sCmMqD
LcTeF01zXHP2sQxFYUPC3R0sXTvxjRmCosGr5Z0eJHQe

Sergio Acosta > echo "==== CLAVES.TXT ===="; cat claves.txt
==== CLAVES.TXT ====
MP2EhDEUSKxSPjxv5-nNDfPH11fNVKELyVn13t002z0pM00zj1v-Ha8xkb-wIU
hQqT3YhWwEtoBwAPaje9R0tI30TWHHP08lGm=M6LDnBaK0kHaMtAvgBhzncccfK
gmVzX2LmTHZD0Qten0w1r6jsX03zLRfsK2lhd775uHSD0WwT/8meXQt0epJn44R
HXUlljzPS7tsBhNvr338DPLSE0/wPMe4cVXH9H4G2Ey/Ey0Zz7D3ESMY24kc5qmN
rBdNwkEhwjoJ0gIVDFI1WmkuMch2qWMPpPrFuMtsADHR96YK11W6R6P40DORTDfx
51mN6DPrz5CZf2P3p0CpUw==

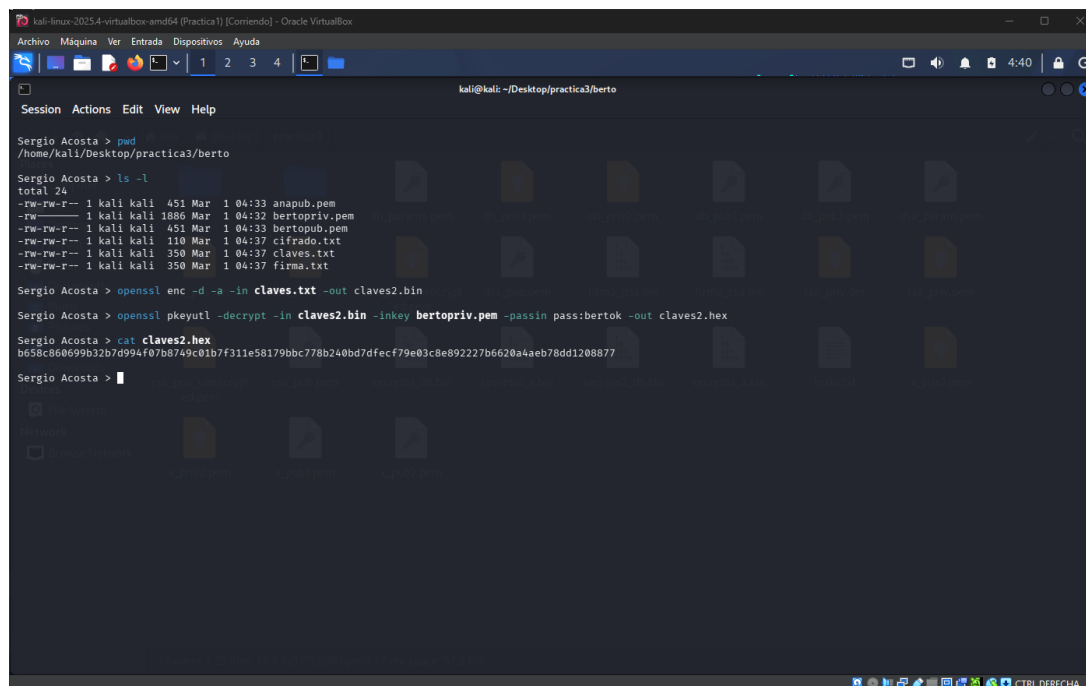
Sergio Acosta > echo "==== FIRMA.TXT ===="; cat firma.txt
==== FIRMA.TXT ====
fG0PDwXnsZf3PPANGdmNPuklf8K70Y+JpL978uhV4vE1uxSc7y5WxvzIISK0W
mCpPH1mkuC00pgFzFs+uPgn7W1rCwajpRzoLSQJNW+U8cE7euG0L25f0h3T+wfK
tZqnc3WxvLLjL1KtJ2B6skV13/URhrtPt134L22HqNAL4B5B3Zuy2m+Hk0AE0
F4EbSMhdmsShePMLfEvLndm31kUgtGOoaMTJWnbBzZuaLXTEJrF3DUUfUaFuk02Mo
ABWoku1Af5U5yo5bS9fxAmUs+dtX/kmCTL594Mp7Tc3qzc8f9BTS169CLOmVVBan
NfW5ID+k1Bkgv65z72W95w==

Sergio Acosta >
```

Figura 3.3: Listado del contenido en BASE64 de los ficheros cifrado.txt, claves.txt y firma.txt.

3.3. Operaciones de Berto (Receptor)

Actuando como Berto dentro de su respectivo directorio, se procede a recuperar los ficheros y a descifrarlos. Como vemos en la figura 3.4, primero se convierte el documento `claves.txt` a formato binario y se descifra empleando la clave privada de Berto. De este modo, se extrae de `claves2.hex` la misma clave y el mismo vector que usó Ana inicialmente.

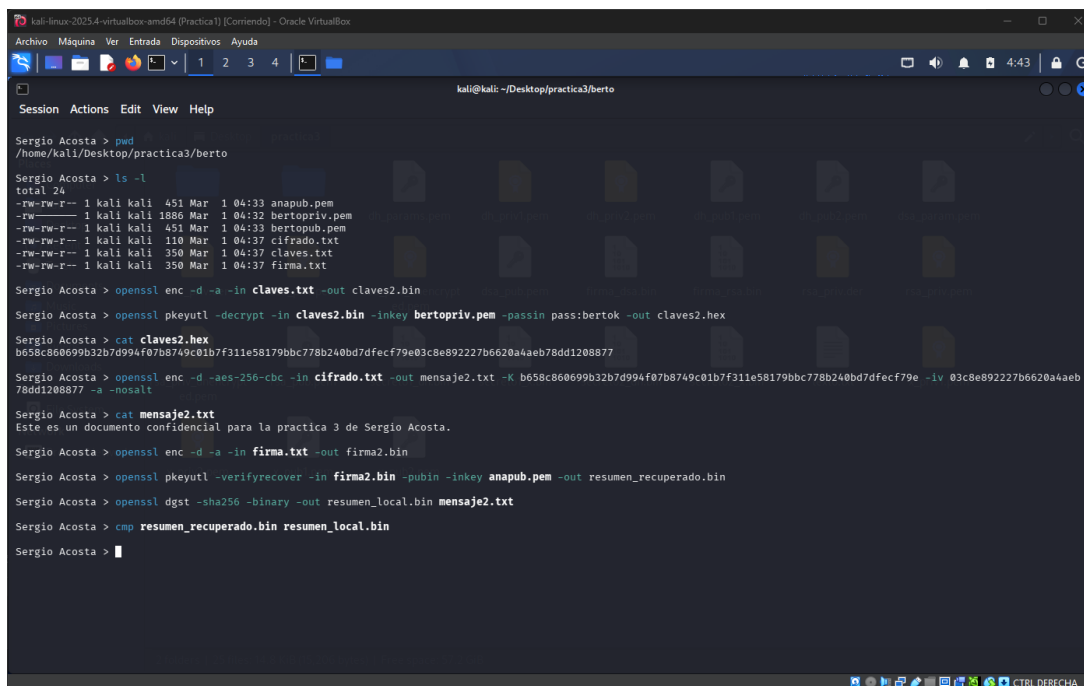


```
kali-linux-2025.4-virtualbox-amd64 (Practica1) [Comando] - Oracle VM VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
kali@kali: ~/Desktop/practica3/berto
Sergio Acosta > pwd
/home/kali/Desktop/practica3/berto
Sergio Acosta > ls -l
total 24
-rw-rw-r-- 1 kali kali 451 Mar 1 04:33 anapub.pem
-rw-rw-r-- 1 kali kali 1886 Mar 1 04:32 bertopriv.pem
-rw-rw-r-- 1 kali kali 451 Mar 1 04:33 bertopub.pem
-rw-rw-r-- 1 kali kali 110 Mar 1 04:37 cifrado.txt
-rw-rw-r-- 1 kali kali 350 Mar 1 04:37 claves.txt
-rw-rw-r-- 1 kali kali 350 Mar 1 04:37 firma.txt
Sergio Acosta > openssl enc -d -a -in claves.txt -out claves2.bin
Sergio Acosta > openssl pkeyutl -decrypt -in claves2.bin -inkey bertopriv.pem -passin pass:bertok -out claves2.hex
Sergio Acosta > cat claves2.hex
b658c860699b32b7d994f07b8749c01b7f311e58179bbc778b240bd7dfecf79e03c8e892227b6620a4eb78dd1208877
Sergio Acosta >
```

Figura 3.4: Descifrado del fichero de claves con la clave privada de Berto para recuperar Key e IV.

Finalmente, como podemos comprobar en la figura 3.5, se usan dichos parámetros simétricos para descifrar `cifrado.txt`, obteniendo como resultado un `mensaje2.txt` idéntico al original.

Para concluir, se verifica la autenticidad del envío utilizando la opción `-verifyrecover` para recuperar el resumen descifrando la firma con la clave pública de Ana. Haciendo uso del comando `cmp`, este resumen se compara con un hash SHA-256 local extraído de `mensaje2.txt`. La ejecución no devuelve ningún error, confirmando de manera sólida la integridad y el origen de los datos.



The screenshot shows a terminal window titled "kali-linux-2023.4-virtualbox-amd64 (Practica1) [Corriendo] - Oracle VM VirtualBox". The user is logged in as Sergio Acosta at the kali machine, with the current directory being /home/kali/Desktop/practica3/berto. The terminal displays the following commands and outputs:

```
Sergio Acosta > pwd
/home/kali/Desktop/practica3/berto

Sergio Acosta > ls -l
total 24
-rw-rw-r-- 1 kali kali 451 Mar 1 04:33 anapub.pem
-rw-rw-r-- 1 kali kali 1886 Mar 1 04:32 bertopriv.pem
-rw-rw-r-- 1 kali kali 451 Mar 1 04:33 bertopub.pem
-rw-rw-r-- 1 kali kali 110 Mar 1 04:37 cifrado.txt
-rw-rw-r-- 1 kali kali 350 Mar 1 04:37 claves.txt
-rw-rw-r-- 1 kali kali 350 Mar 1 04:37 firma.txt

Sergio Acosta > openssl enc -d -a -in claves.txt -out claves2.bin

Sergio Acosta > openssl pkeyutl -decrypt -in claves2.bin -inkey bertopriv.pem -passin pass:bertok -out claves2.hex

Sergio Acosta > cat claves2.hex
b658c860699b32b7d994f07b8749c01b7f311e58179bbc778b240bd7dfecf79e03c8e892227b6620a4aeb78dd1208877

Sergio Acosta > openssl enc -d -aes-256-cbc -in cifrado.txt -out mensaje2.txt -K b658c860699b32b7d994f07b8749c01b7f311e58179bbc778b240bd7dfecf79e -iv 03c8e892227b6620a4aeb78dd1208877 -a -nosalt

Sergio Acosta > cat mensaje2.txt
Este es un documento confidencial para la practica 3 de Sergio Acosta.

Sergio Acosta > openssl enc -d -a -in firma.txt -out firma2.bin

Sergio Acosta > openssl pkeyutl -verifyrecover -in firma2.bin -pubin -inkey anapub.pem -out resumen_recuperado.bin

Sergio Acosta > openssl dgst -sha256 -binary -out resumen_local.bin mensaje2.txt

Sergio Acosta > cmp resumen_recuperado.bin resumen_local.bin

Sergio Acosta >
```

Figura 3.5: Descifrado final del texto plano y verificación exitosa de los resúmenes.

Bibliografía

- [1] The OpenSSL Project, “OpenSSL Documentation,” [En línea]. Disponible: <https://www.openssl.org/docs/man3.2/index.html>.
- [2] CICEI, “OpenSSL Howto,” [En línea]. Disponible: <https://openssl.cicei.com>.
- [3] Wikipedia, “Block cipher mode of operation,” [En línea]. Disponible: https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation.
- [4] Wikipedia, La enciclopedia libre, [En línea]. Disponible: <https://wikipedia.com>.
- [5] OpenAI, “ChatGPT,” Herramienta de IA, [En línea]. Disponible: <https://openai.com/chat>.